C Spreadsheet Project

Swara, Ayachi

1 Introduction

Brief description of what the project does.

This project is a command-line spreadsheet written in C, similar to Microsoft Excel but with a command-line interface. A grid of cells is created by the program, where each cell stores an integer value. Users can interact with the spreadsheet by typing commands: they can write cell values directly or provide references to other cells and perform arithmetic operations.

The program is launched using:

```
./sheet r c
```

where `r` is the number of rows and `c` is the number of columns.

At any given instant, only a 10×10 portion of the sheet is visible to the user. Scroll commands allow the user to move to different parts of the sheet. The program keeps accepting commands and executing them until the user enters `q` to exit.

2 Challenges and Design Decisions

In developing this spreadsheet, we encountered several architectural challenges. Below we detail the engineering trade-offs and design decisions made to ensure the program remains performant and robust.

2.1 Memory Optimization and Performance (Chunking)

A major challenge was the potential for extremely high memory usage and slow execution times when handling the maximum allowed grid size (ZZZ999).

- **The Challenge:** Allocating a flat 2D array for millions of cells would lead to a massive memory footprint, most of which would likely be empty (sparse).
- **The Solution:** We implemented a **Chunked Memory Management** system. Instead of allocating all cells upfront, we use a two-level pointer system that allocates "chunks" of 1024 cells only when a cell within that range is accessed.
- **Why this works:** This approach is highly compatible with **L1/L2 Caching**. By keeping 1024 cells in a contiguous block, the CPU can load a chunk into cache memory once, making operations on neighboring cells significantly faster due to spatial locality.

2.2 Dependency Tracking and Graph Structure

Spreadsheets are essentially Directed Graphs. Tracking which cells depend on each other was a core difficulty because a single cell might have multiple parents (dependencies) and multiple children (dependents).

- **Design Decision:** We included both a `dependencies` list and a `dependents` list in our `Cell` struct.
- **The Benefit:** Keeping both lists allows for bidirectional traversal. We use the `dependencies` list during formula evaluation to know what values to fetch, and we use the `dependents` list to "push" updates forward when a cell's value changes.

2.3 Circular Formula Detection

One of the most dangerous edge cases in a spreadsheet is a circular reference (e.g., $A1 = B1 + 1$ and $B1 = A1 + 1$).

- **Our Approach:** We chose to perform cycle detection **at the moment of assignment**.
- **Logic:** Before finalizing a new formula, we run a Depth First Search (DFS) using the `visited` and `in_stack` flags. If we encounter a node already in the current recursion stack, we identify a back-edge (a cycle). This allows us to block the operation immediately and alert the user rather than letting the program enter an infinite loop.

2.4 Push Model vs. Dirty Flagging

We debated between using "Dirty Flagging" (marking cells as invalid and recalculating only when viewed) and "Immediate Evaluation" (recalculating everything affected by a change).

- **Decision:** We opted for **Immediate Evaluation** (the "Push" model).
- **Reasoning:** For standard academic or personal use-cases, it is rare for a single cell to have tens of thousands of direct dependents. The complexity of managing a dirty-flagging state machine was higher than the performance cost of a recursive push. This ensures the grid is always in a consistent state and simplifies the rendering logic.

2.5 Stale Edges and Error Propagation

When a user overwrites an old formula, there is a risk of "Stale Edges"—where the cell still thinks it depends on its previous parents.

- **Solution:** We implemented a strict `clear_dependencies()` function that runs before every formula update. It disconnects the cell from its previous parents' `dependents` lists, effectively "wiping" the old graph edges before drawing new ones.
- **ERR Propagation:** If a calculation results in an error (like Division by Zero), the cell is marked with an `is_err` flag. This flag "propagates virally." When a child cell recalculates, it checks if any of its parents are in an `ERR` state; if they are, the child also evaluates to `ERR` and passes that state down the chain.

3 System Organisation

The spreadsheet is designed using a modular architecture to separate concerns between memory management, mathematical logic, and graph-based recalculation. This structure ensures that each component can be tested and optimized independently.

3.1 File Breakdown and Responsibilities

- **common.h**: This is the core header file that defines the shared "Global State" of the program. It contains the primary data structures:
  - `Cell`: Stores the numerical value, the original formula string, error flags, and adjacency lists for the dependency graph.
  - `Grid`: Manages the dimensions of the sheet and the chunked row pointers.
  - `DependencyNode`: A linked-list structure used to track parent-child relationships between cells.
- **main.c**: Acts as the entry point and the system orchestrator.
  - **REPL Loop:** Implements the Read-Eval-Print loop that waits for user commands.
  - **Command Routing:** Parses the input and routes it to the correct subsystem (e.g., scrolling, formula assignment, or quitting).
  - **Execution Timer:** Uses `clock_gettime` to measure and display the latency of the last operation.
- **grid.c**: Manages the physical representation and storage of the grid.

- **Chunked Allocation:** Implements the `get_cell` function, which lazily allocates 1024-cell chunks to optimize memory and L1/L2 cache performance.
- **View Management:** Handles the 10×10 window logic, translating relative scroll coordinates into absolute grid indices.

- **parser.c**: The mathematical engine of the spreadsheet.
  - **Expression Evaluation:** Uses the Shunting-Yard algorithm and two stacks (operators and values) to handle operator precedence and parentheses.
  - **Range Logic:** Implements `evaluate_range` to handle functions like `SUM`, `AVG`, and `STDEV` across 1D and 2D cell blocks.
- **engine.c**: Manages the logic of the Directed Acyclic Graph (DAG).
  - **Dependency Registration:** Links parent cells to children during formula parsing.
  - **Cycle Detection:** Uses a Depth-First Search (DFS) with a recursion stack to prevent infinite loops.
  - **Recursive Propagation:** Implements the "Push Model" where changing a single cell triggers an automated ripple-effect update to all its dependents.

3.2 Build and Automation

- **Makefile**: Automates the compilation of individual objects and links them with the math library (`-lm`). It provides specific targets for building the executable (`make`), running the test suite (`make test`), and generating the PDF report (`make report`).
- **test_suite.c**: A standalone program that executes the spreadsheet binary as a child process. It pipes a series of edge-case commands (like circular references and complex range functions) and verifies the output to ensure system stability.

4 Data Structures

The efficiency of our spreadsheet relies on three primary data structures that work together to manage memory, store data, and track relationships between cells.

4.1 The Cell Struct: The Atom of the Spreadsheet

The `Cell` struct is the most vital component. It doesn't just store a value; it acts as a node in a massive network.

- **Storage:** It holds a `double` for precision and a `char*` to store the original formula string.
- **Graph Flags:** It contains `visited` and `in_stack` booleans. These are essential for our DFS-based cycle detection, ensuring we don't crash when a user types $A1 = A1$.
- **Adjacency Lists:** Each cell maintains two pointers to `DependencyNode` linked lists: `dependents` (cells that rely on this one) and `dependencies` (cells this one relies on).

4.2 The DependencyNode: Linking the Graph

To handle a variable number of connections without wasting memory, we used a **Singly Linked List** of `DependencyNode` objects.

- Instead of using a fixed-size array (which would limit how many cells can reference another), the linked list allows $B1$ to be referenced by an unlimited number of other cells.
- Each node simply contains a pointer to the related `Cell` and a pointer to the `next` node in the list.

4.3 The Grid Struct: Hierarchical Memory

The `Grid` struct manages the overall dimensions and the "view" into the spreadsheet.

- **Three-Level Pointer System:** We use `Cell*** rows`. This is a pointer to an array of row pointers, where each row pointer points to a **Chunk** of cells.
- **Lazy Allocation:** By using this hierarchical structure, we avoid allocating millions of `Cell` objects at startup. Memory for a chunk of 1024 cells is only allocated the first time a user interacts with a cell in that range.
- **State Tracking:** It stores `scroll_row` and `scroll_col` to manage which 10×10 window is currently being rendered to the terminal.

5 Edge Cases and Test Suite

To ensure the reliability of our spreadsheet, we developed a comprehensive automated testing framework in `test_suite.c`. This suite validates the program's behavior across arithmetic, graph logic, and system commands.

5.1 Automated Testing Framework

Rather than relying on manual keyboard input, our test suite uses a **Parent-Child Process Model** utilizing UNIX pipes:

- **Simulation:** The suite forks a child process to run the `./sheet` binary. The parent process acts as the "user," writing commands to the child's `stdin` via a pipe.
- **Verification:** The parent reads the child's `stdout` and uses substring matching to verify that the expected values appear in the grid.
- **Efficiency:** This allows us to run dozens of complex regression tests in milliseconds, ensuring that new code changes do not break existing functionality.

5.2 Key Edge Cases Tested

Our suite specifically targets the following critical scenarios:

- **Mathematical Precedence:** We verified that expressions like `A1=2+3*4` correctly evaluate to `14` rather than `20`, confirming our Shunting-Yard implementation respects BODMAS rules.
- **Circular Dependency Detection:** We tested both self-references ($A1 = A1$) and mutual references ($A1 = B1, B1 = A1$). The suite confirms that the program identifies the cycle and marks the cell as `ERR` instead of entering an infinite recursion.
- **Viral Error Propagation:** We tested "chained errors" where if $A1$ is `ERR` (e.g., due to division by zero), any cell referencing it (e.g., $B1 = A1 + 5$) also automatically switches to an `ERR` state.
- **Boundary Conditions:** We validated assignments to the extreme edges of the grid (e.g., $ZZZ999$) and verified that range functions like `SUM(A1:ZZZ999)` handle large datasets without memory corruption.
- **Precision and Truncation:** We verified that while internal math is handled in `double` precision (crucial for `AVG` and `STDEV`), the final display correctly truncates values to integers per the lab requirements.

5.3 System Command Validation

The suite also automates the testing of our custom helper functions:

- **Timing Logic:** Verified that using `SLEEP(n)` correctly updates the execution timer shown in the prompt.
- **Navigation:** Confirmed that `scroll_to` commands correctly shift the 10×10 viewing window to the intended coordinates.

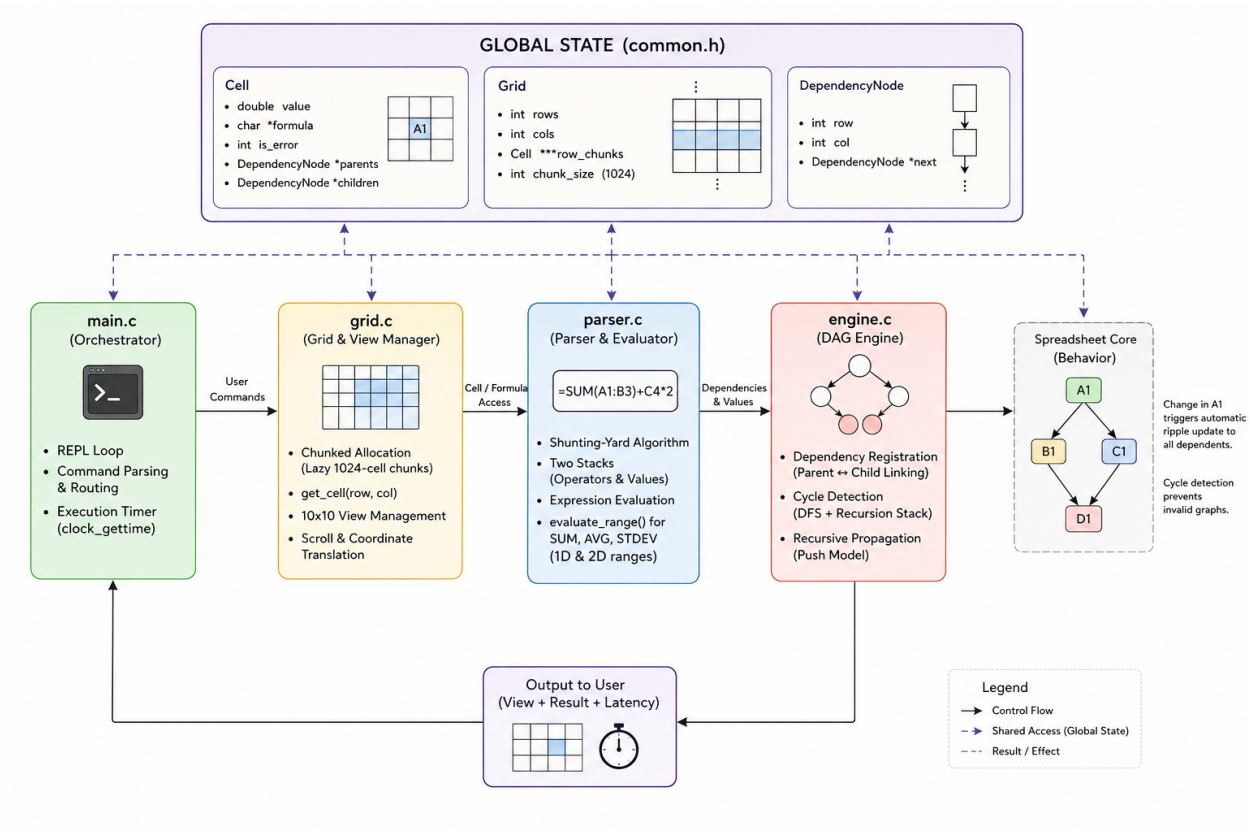

Figure 1: Spreadsheet System Architecture and File Dependencies

6 GitHub Links

GitHub Repository: Ayachi_Swara_C_Project
Commit History: Project Commit History